



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Castro Blanco Manuel David

Fecha:
04 de Mayo 2026

Este documento presenta el análisis completo para el desarrollo de un sistema web que permita gestionar asesorías académicas entre alumnos y profesores. Se incluyen requisitos funcionales y no funcionales, modelado mediante casos de uso y flujo de proceso, selección y justificación de metodología ágil, elección de tecnologías, identificación de problemas potenciales, mejoras futuras y patrones de diseño aplicados.

FUNCIONALES

Registro de alumnos	El sistema debe permitir que un alumno cree una cuenta proporcionando su nombre completo, número de matrícula, correo institucional y contraseña.
Inicio de sesión (alumnos y profesores)	El sistema debe autenticar a alumnos y profesores mediante correo institucional y contraseña. Cada rol tendrá acceso únicamente a las vistas y funciones que le corresponden.
Solicitud de asesoría por parte del alumno	El alumno autenticado debe poder enviar una solicitud de asesoría indicando: materia, tema o duda específica, y fecha/horario preferido.
Consulta del estado de solicitudes (alumno)	El alumno debe poder visualizar el historial de sus solicitudes junto con su estado actual: Pendiente, Aceptada o Rechazada. También debe poder ver la fecha y el motivo de rechazo en caso de que aplique.
Visualización y gestión de solicitudes (profesor)	El profesor autenticado debe poder ver el listado de solicitudes que le han sido asignadas, con sus detalles completos.

NO FUNCIONALES

Seguridad	Las contraseñas deberán almacenarse cifradas. Las sesiones estarán protegidas mediante tokens.
Usabilidad	La interfaz debe ser intuitiva y responsiva (adaptada a móviles y escritorio).
Disponibilidad y rendimiento	El sistema debe estar disponible al menos el 99% del tiempo durante el ciclo escolar. El tiempo de respuesta para operaciones comunes no debe superar 2 segundos bajo carga normal.

METODOLOGIA

Yo usaria SCRUM para dividir todas las funcionalidades en sprints con tiempos de entregas concretas. Se puede ajustar con nuevos cambios o requisitos por ejemplo en personal DOCENTE pudiendo realizar entregas incrementales.

Modalidad de trabajo: En equipo (3-4 integrantes)

Si bien el proyecto puede desarrollarse individualmente, trabajar en equipo permite realizar multiples tareas, mejorar la calidad mediante revisión entre pares y reducir los tiempos de entrega. La división de roles propuesta sería:

Líder	Coordina sprints, elimina impedimentos, da seguimiento.
Backend	API REST, lógica de negocio, autenticación, base de datos.
Frontend	Interfaz de usuario, responsividad, formularios, consumo de API.
QA / Documentador	Pruebas funcionales, documentación técnica y de usuario.

Frontend	CSS	Diseño desde 0
Backend	PHP JS	Entorno JavaScript del lado del servidor con amplio ecosistema de paquetes. APIs REST y PHP para logica de negocio y servidor.
Base de Datos MySQL		Base de datos relacional, ideal para gestionar solicitudes con estados y relaciones claras entre alumnos.
Control de Versiones	Git + GitHub	Permite el trabajo colaborativo, historial de cambios y vinculación con el repositorio requerido como entregable del caso práctico.
Despliegue	Hostinger / AWS	Plataformas para mantener ONLINE sistemas web y otro tiene particularidad de manejar espejo-servidor.

Problemas Potenciales

Conflictos de horario: Dos alumnos podrían solicitar asesoría al mismo profesor en el mismo horario. Si el sistema no valida la disponibilidad en tiempo real, pueden generarse solicitudes aceptadas. Solución: Implementar un módulo de disponibilidad donde el profesor registre sus horarios libres y el sistema bloquee horarios ya ocupados al momento de aceptar una solicitud.

Abandono de solicitudes sin respuesta: Un profesor podría no responder solicitudes durante días, generando frustración en el alumno. Solución: Establecer un tiempo límite de respuesta. Si se supera, la solicitud se marca como 'Expirada' y el alumno puede reasignarla a otro profesor.

Seguridad en la autenticación: Sin medidas adecuadas, el sistema podría ser vulnerable a ataques. Solución: Implementar rate-limiting en el endpoint de login con tiempo de expiración.

Mejoras Futuras

Notificaciones por correo electrónico o WhatsApp cuando el estado de una solicitud cambie.

Panel de administración institucional con métricas: solicitudes por materia, tiempos de respuesta promedio, profesores con más carga.

Integración con el sistema de calificaciones para identificar alumnos con bajo rendimiento y sugerirles proactivamente una asesoría.

MVC (Modelo Vista Controlador)

Separa las responsabilidades del sistema en tres capas: Modelo (datos y lógica como Solicitud, Alumno, Profesor), Vista y Controlador (endpoints que gestionan las peticiones).

